

Nube y Data Lakes para MLOps

Almacenamiento descentralizado: dónde viven datos, modelos y artefactos —
y cómo alimenta a los protocolos que ya vimos.

Data Lake vs Data Warehouse

MinIO / S3

Airflow · MLflow · Serving

Sesión 6 · incluye Hito TP #2 (checkpoint del integrador)

Agenda

- 1 El dato hoy** desafíos: calidad, obsolescencia, volumen, costo
- 2 Data Warehouse vs Data Lake** top-down vs bottom-up · ETL vs ELT
- 3 Beneficios y riesgos** por qué un lake para ML · el 'data swamp' · Lakehouse
- 4 Lakes en la nube** S3, ADLS, GCS · MinIO local · despliegue gestionado
- 5 El lake en la plataforma** Airflow + MLflow + serving desde S3/MinIO
- 6 El lake y los protocolos vistos** streaming · gRPC · GraphQL · federado
- 7 Práctica + Mini-TP 6 + Hito TP #2** tutorial datalake y checkpoint del integrador

¿Dónde estamos?

Ya sabemos cómo se hablan los servicios y cómo fluye el dato en movimiento. Falta lo de siempre: ¿dónde vive el dato en reposo, y los modelos y artefactos que produce la plataforma?

Lo que ya vimos (en movimiento)

- REST · GraphQL · gRPC — cómo se piden y sirven predicciones
- Streaming (Kafka) — eventos que fluyen en tiempo real
- Federado — el modelo viaja al dato, se comparten pesos

Lo que agregamos hoy (en reposo)

- **Data Lake** — dónde aterriza y se guarda **TODO** el dato crudo
- Artefactos y modelos versionados (MLflow → MinIO/S3)
- La nube como sustrato: elasticidad, costo, gobernanza

REST

GraphQL

gRPC

Streaming

Federado

Cloud/Lake

Seguridad

Integra

El dato hoy: cinco dolores

Calidad

El 60% reporta la calidad como su desafío #1. Datos sucios → modelos sucios.

Obsolescencia

El 86% usó datos desactualizados alguna vez. El dato caduca.

Volumen

Crece más rápido que cualquier esquema rígido que intentemos imponerle.

Escala

Series temporales, logs, imágenes, documentos: formatos que un DW no traga.

Costo

Almacenar todo en un warehouse es caro; tirar dato hoy es perder valor mañana.

Data Warehouse: enfoque top-down

Primero el esquema, después el dato

- Empieza por los requisitos de negocio y los reportes deseados.
- Se define el esquema y las consultas ANTES de cargar el dato.
- ETL: extraer → transformar → cargar ya estructurado y limpio.
- **Schema-on-write: el dato entra solo si encaja en el modelo.**
- Optimizado para SQL/BI: consultas rápidas, 'una versión de la verdad'.
- Fuentes relacionales, volúmenes y formatos conocidos.

Fortalezas y límites

- ✓ Excelente para reportes y analítica de negocio (BI).
- ✓ Gobernanza y consistencia fuertes.
- ✗ No admite datos no estructurados / semiestructurados.
- ✗ Transformaciones rígidas: no siguen el ritmo del volumen.
- ✗ Alto costo de cómputo y almacenamiento acoplados.
- ✗ **Para ML muchas veces llegás tarde: el dato crudo ya se perdió.**

Data Lake: enfoque bottom-up

Primero el dato, el esquema al leer

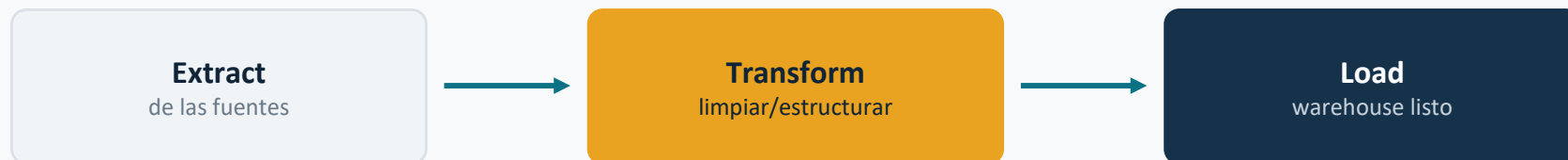
- Ingesta **TODO** el dato, sin importar el requisito futuro.
- Se guarda en formato nativo, sin definir esquema por adelantado.
- ELT: extraer → cargar → transformar SOLO cuando se necesita.
- **Schema-on-read: el esquema se aplica al consumir, no al guardar.**
- Almacenamiento barato y desacoplado del cómputo (S3, HDFS).
- Analítica con motores por encima: Spark, ML, consultas ad-hoc.

Por qué le sirve al ML

- ✓ **Guarda el dato CRUDO: se pueden crear features nuevas después.**
- ✓ Cualquier formato: tabular, texto, imágenes, audio, logs, eventos.
- ✓ Escala barata: separar storage de compute abarata todo.
- ✓ Un único lugar para datasets, artefactos y modelos.
- ⚠ **Riesgo: sin gobernanza se vuelve un 'data swamp' (lo vemos).**

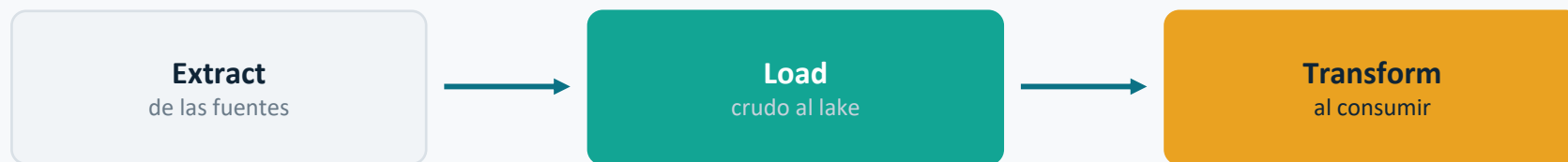
ETL vs ELT: cuándo transformar

ETL — transformar ANTES de guardar (Data Warehouse)



El dato entra ya modelado. Rígido, gobernado, caro.

ELT — transformar DESPUÉS, al leer (Data Lake)



El dato crudo queda disponible; se transforma a demanda. Flexible y barato.

Data Lake vs Data Warehouse

Criterio	Data Lake	Data Warehouse
Dato	Crudo, cualquier formato	Procesado, estructurado
Esquema	Al leer (schema-on-read)	Al escribir (schema-on-write)
Proceso	ELT	ETL
Usuarios	Científicos de datos, ML	Analistas de negocio, BI
Costo	Bajo (storage barato)	Alto (compute acoplado)
Flexibilidad	Alta: explorás, reprocesás	Baja: rígido pero consistente
Mejor para	ML, exploración, dato crudo	Reportes, KPIs, 'la verdad'

¿Por qué un Data Lake para MLOps?

Un solo lugar

Datasets, features, artefactos y modelos conviven. Un origen de verdad para toda la plataforma.

Dato crudo intacto

Se reentrena, se crean features nuevas y se audita con el dato original. No se tira nada.

Barato y elástico

Storage de objetos separado del compute: se paga por lo que se guarda y se escala sin migrar.

Cualquier formato

Tabular, imágenes, texto, audio, eventos de streaming: todo aterriza en el mismo lake.

Reproducibilidad

Versionás datasets y modelos por ruta/tag: un experimento se puede repetir exacto.

Integra con todo

Airflow escribe, MLflow guarda, el serving lee. Y los protocolos vistos se apoyan en él.

El riesgo: del lake al 'data swamp'

La libertad del lake se paga: sin disciplina, un data lake se convierte en un pantano donde nadie encuentra nada ni confía en nada.

Síntomas del pantano

- Datos duplicados, sin dueño y sin documentación.
- No se sabe qué es fresco y qué está obsoleto.
- Fiabilidad rota: múltiples copias, ninguna canónica.
- Sin control de acceso: todo o nada.
- Arquitectura compleja difícil de operar.

Antídotos (buenas prácticas)

- **Zonas: raw → staged → curated (bronze/silver/gold).**
- Convención de rutas/particiones y catálogo de metadatos.
- Formatos columnar (Parquet) para lo curado.
- Versionado y control de acceso por bucket/prefijo.
- Ciclo de vida: expirar lo que no se usa.

Lakehouse: lo mejor de ambos

El Lakehouse añade la gestión y el rendimiento del warehouse (transacciones, versionado, esquema) sobre el almacenamiento barato y abierto del lake. Una sola plataforma para BI y ML.

BI · SQL · Machine Learning · Streaming
una sola plataforma para todos los usos

Gestión: transacciones ACID · versionado · esquema · gobernanza
capa que el lake clásico no tenía

Data Lake abierto: S3 / ADLS / GCS (formatos nativos, barato)
el sustrato de siempre

Ejemplos: Delta Lake, Apache Iceberg, Apache Hudi · Databricks · Snowflake

Data Lakes en la nube

AWS

Amazon S3 como núcleo. Glue (ETL), Athena (SQL sobre S3), Lake Formation (gobierno), Redshift (DW).

Azure

Azure Data Lake Storage (ADLS) Gen2. Synapse, Databricks, Data Factory para orquestar.

Google Cloud

Google Cloud Storage (GCS). BigQuery, Dataproc/Dataflow para procesar.

¿Por qué la nube y no on-premise?

- Elasticidad: el storage y el compute escalan a demanda, sin comprar hardware.
- Menos costo de mantenimiento e infraestructura; foco en extraer valor del dato.
- Seguridad y gobernanza gestionadas; alta disponibilidad y durabilidad del dato.

MinIO: tu Data Lake compatible con S3, en local

MinIO habla el mismo protocolo que Amazon S3. El código escrito contra MinIO en la máquina local corre igual contra S3 en la nube: solo cambian el endpoint y las credenciales.

Conceptos S3 que usamos

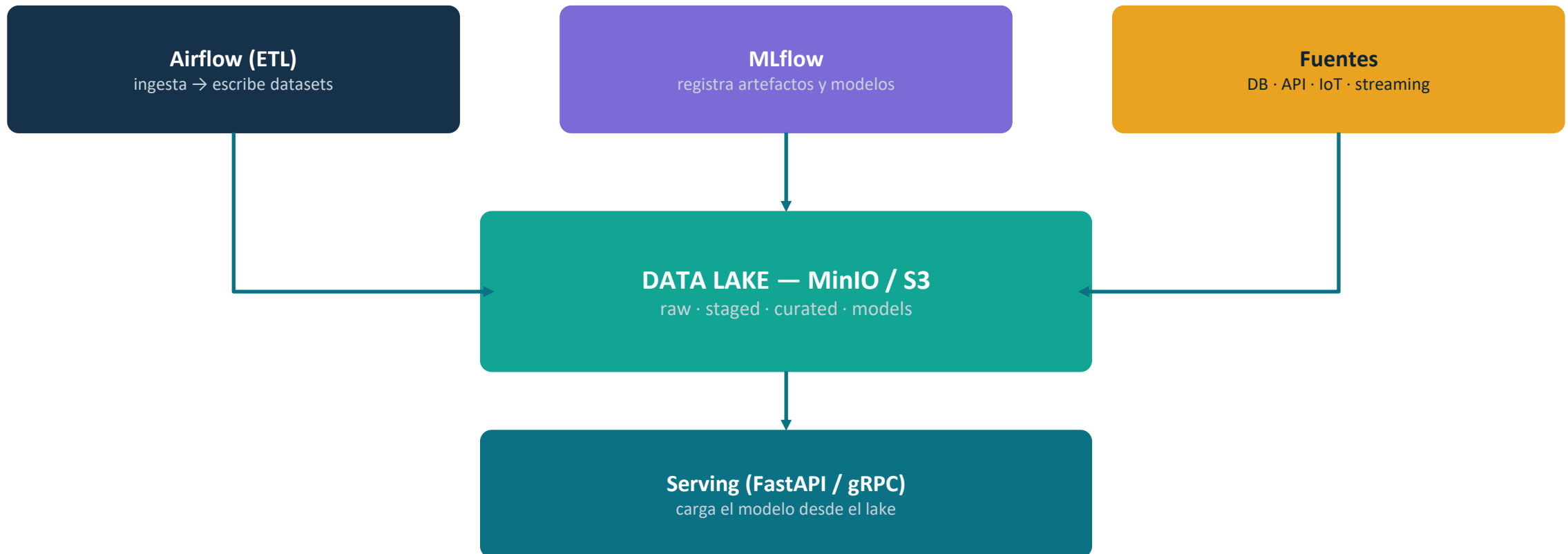
- Bucket — el contenedor de más alto nivel ($\approx$ un lake o zona).
- Object (key) — el archivo y su ruta completa dentro del bucket.
- Prefijo — carpetas lógicas: sirven de partición (año=/mes=).
- Formato — Parquet/CSV/JSON; Parquet para lo curado.
- Endpoint + access/secret key — así se realiza la conexión (boto3).

```
# mismo código para MinIO local y S3 en la nube

import boto3
s3 = boto3.client(
    "s3",
    endpoint_url="http://localhost:9000",
    aws_access_key_id="minio",
    aws_secret_access_key="minio123",
)
s3.create_bucket(Bucket="datalake")
s3.upload_file("modelo.pkl",
    "datalake", "models/v1/modelo.pkl")
```

El Data Lake dentro de la plataforma

El lake es el piso común: Airflow escribe datos, MLflow guarda artefactos, el servicio lee el modelo — todos contra el mismo S3/MinIO.

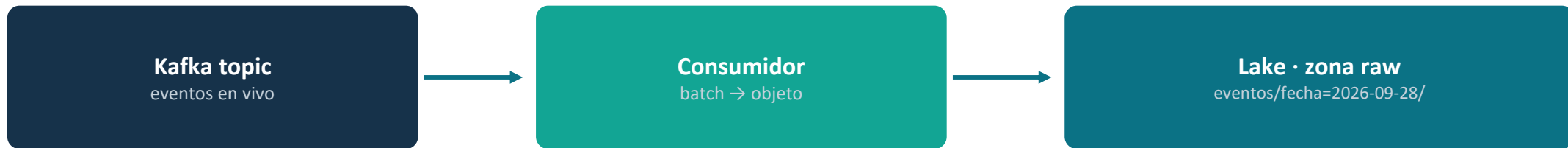


El Data Lake y los protocolos vistos

Protocolo	Rol del Data Lake
Streaming (Kafka)	Los eventos del stream ATERRIZAN en la zona raw del lake; el consumidor persiste cada mensaje como objeto (particionado por fecha). El lake es la memoria durable del stream.
gRPC	El servicio de inferencia CARGA su modelo desde el lake al arrancar; puede leer features precalculadas de la zona curated para responder con baja latencia.
GraphQL	La API CONSULTA metadatos, features y predicciones almacenadas en el lake, resolviendo cada campo del grafo contra el mismo origen de verdad.
Federado	El modelo global se VERSIONA en MinIO/MLflow tras cada ronda; se distribuye a los clientes desde el lake y se guardan las métricas de cada ronda.

Streaming → Data Lake

Un stream es efímero por diseño (retención corta). El lake le da memoria: el consumidor escribe cada evento como objeto, particionado por fecha, para reentrenar y auditar después.



```
# el stream fluye; el lake recuerda

# consumidor Kafka que persiste al lake
for msg in consumer:
    evento = json.loads(msg.value)
    buffer.append(evento)
    if len(buffer) >= 500:
        key = f"eventos/fecha={hoy}/parte-{n}.parquet"
        s3.put_object(Bucket="datalake", Key=key,
                      Body=to_parquet(buffer))
    buffer.clear()
```

gRPC y GraphQL sobre el Data Lake

gRPC — servir cargando del lake

- Al arrancar, el servicio descarga el modelo desde S3/MinIO.
- No hay modelo 'horneado' en la imagen: se versiona en el lake.
- Actualizar modelo = subir una versión nueva al lake y reiniciar.

```
# servidor gRPC

s3.download_file("datalake",
    "models/v3/modelo.pkl", "/tmp/m.pkl")
model = joblib.load("/tmp/m.pkl")
# servir por gRPC con el modelo del lake
```

GraphQL — consultar el lake

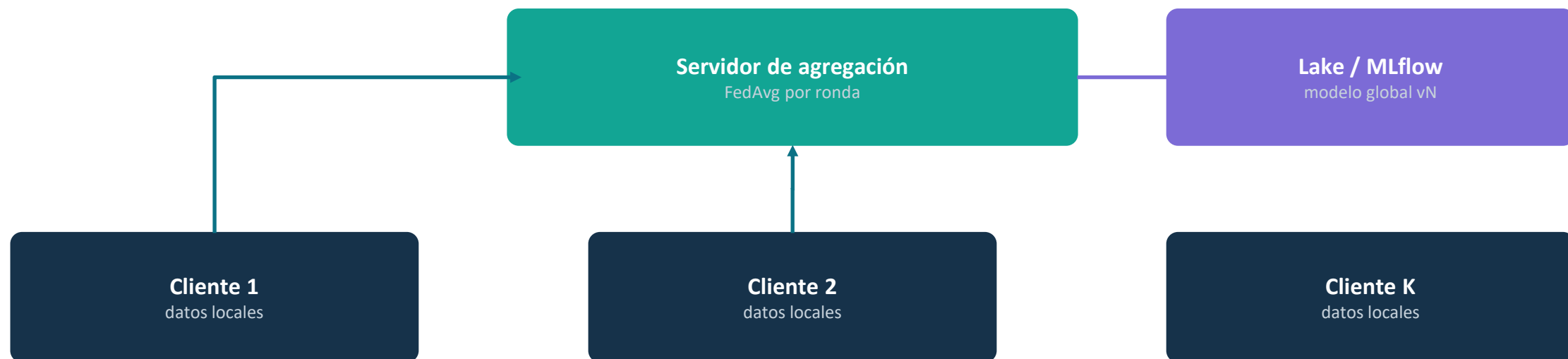
- Cada campo del grafo resuelve contra el mismo origen: el lake.
- Features precalculadas y predicciones viven en la zona curated.
- El cliente pide solo lo que necesita; el resolver lee del lake.

```
# resolver GraphQL

def resolve_prediccion(id):
    key = f"curated/preds/{id}.json"
    obj = s3.get_object(Bucket="datalake",
                        Key=key)
    return json.load(obj["Body"])
```

Federado + Data Lake

En federado el dato NO se centraliza, pero el modelo global sí necesita un hogar: el lake versiona cada ronda y distribuye el modelo a los clientes.



↑ pesos (nunca datos) → cada ronda versiona el modelo global en el lake

Despliegue gestionado en la nube

Amazon SageMaker

Entrenamiento, registro y endpoints gestionados sobre S3.

Google Vertex AI

Pipelines, registro de modelos y serving sobre GCS.

Azure ML

Workspaces, datasets y endpoints sobre ADLS.

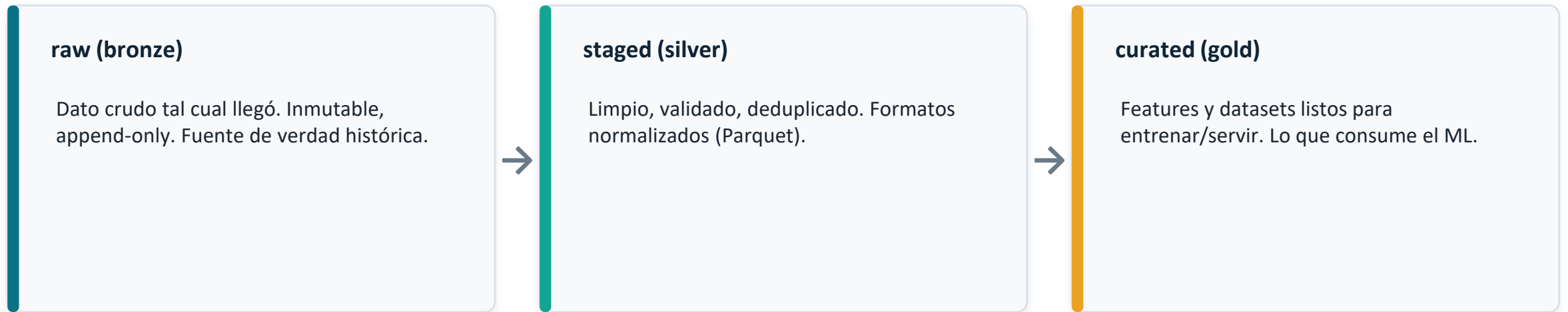
A favor

- Menos infra que operar: escalado, monitoreo y seguridad incluidos.
- Del experimento al endpoint en pocos pasos.
- Integración nativa con el storage del proveedor.

A tener en cuenta

- Costo: cómodo pero puede escalar rápido; medir siempre.
- Lock-in: el pipeline queda atado a un proveedor.
- Menos control fino que un stack propio (Airflow+MLflow+MinIO).

Zonas del lake: raw → staged → curated



Regla práctica: nunca se borra la zona raw; se transforma hacia curated. Cada modelo apunta a una versión concreta de curated para ser reproducible.

Práctica: mapa de la sesión

Levantamos MinIO con Docker y lo usamos como Data Lake S3: creamos zonas, subimos un dataset y un modelo, y servimos cargando desde el lake. Integramos con MLflow.

- 1 Levantar MinIO** `docker run ... minio server`
- 2 Crear bucket y zonas** `raw / staged / curated / models`
- 3 Subir dataset y modelo** `boto3 put/upload`
- 4 MLflow → MinIO** `artefactos y modelo versionados`
- 5 Servir desde el lake** `descargar el modelo y predecir`

 `clase6/Teoria/datalake_tutorial.ipynb`

Errores comunes

Al trabajar con el lake

- Preparar el modelo en la imagen en vez de leerlo del lake.
- Guardar todo en raw y nunca curar → data swamp.
- CSV para todo: conviene Parquet en lo curado (peso y velocidad).
- No particionar: un prefijo plano con millones de objetos.
- Credenciales hardcodeadas en el código (mejor usar variables de entorno).

Al integrar y desplegar

- Olvidar que MinIO y S3 comparten API: probar en local, subir a la nube.
- No versionar el modelo: sin models/v1, v2 no hay rollback.
- MLflow apuntando a disco local en vez de a MinIO/S3.
- Ignorar el costo: medir tamaño y ciclo de vida del dato.
- Endpoint mal configurado (http vs https, puerto 9000).

Mini-TP 6 + Hito TP #2

Mini-TP 6 (individual)

- Subir el artefacto del modelo propio a MinIO/S3.
- Servir el modelo cargándolo desde el lake (no desde disco).
- Organizar el bucket con zonas y versión (models/v1/...).
- (Opcional) Registrar el experimento con MLflow → MinIO.
- Starter: mini_tp6_actividad.ipynb

Hito TP #2 — checkpoint del integrador

- Checkpoint EVALUADO EN CLASE del proyecto integrador.
- Mostrar el avance: modelo servido por REST/GraphQL/gRPC.
- Dato y artefactos en un lake (MinIO/S3), no en disco suelto.
- Arquitectura y repo del grupo al día.
- Es el segundo hito: prepara la defensa final (Sesión 8).

Cierre y qué sigue

Lo que queda

- El Data Lake es el piso común: dato crudo, artefactos y modelos en un solo lugar barato y escalable.
- DL vs DW: schema-on-read/ELT vs schema-on-write/ETL; el Lakehouse busca lo mejor de ambos.
- MinIO = S3 en local: el mismo código sirve para practicar y para producción en la nube.
- **Todo lo visto se apoya en el lake: streaming aterriza, gRPC/GraphQL sirven y consultan, federado versiona.**

Próxima sesión → Seguridad en MLOps (autenticación, secretos, superficie de ataque).

REST

GraphQL

gRPC

Streaming

Federado

Cloud/Lake

Seguridad

Integra

Referencias

- MinIO — documentación y compatibilidad S3: min.io/docs
- AWS S3 / boto3 — guía del SDK de Python
- MLflow — Artifact Stores (S3/MinIO): mlflow.org/docs
- Apache Airflow — providers de Amazon (S3): airflow.apache.org
- Delta Lake / Apache Iceberg — formatos de Lakehouse
- **Repositorio del curso — [clase6/Teoria/](#) y [clase6/Practica/](#)**